

Министерство образования и науки Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Кафедра ЦПиС ПИР

Выпускная квалификационная работа бакалавра

Изучение вариантов обеспечения устойчивости ResNet-подобных моделей
компьютерного зрения к небольшим сдвигам входного изображения

Автор:

Полев Алексей Михайлович

Научный руководитель:

Самосюк Алексей Владимирович



Москва 2025

АННОТАЦИЯ

Исследована проблема пространственной инвариантности (shift-invariance) в CNN и её влияние на стабильность алгоритмов компьютерного зрения. Проанализированы артефакты при субпиксельных сдвигах изображений, снижающие надежность систем классификации и детекции.

Формализована проблема отсутствия инвариантности к сдвигам в CNN, проанализированы причины, связанные с субдискретизацией и нарушением теоремы Найквиста-Шеннона. Предложена математическая модель алиасинга при пулинге и страйдинге.

Проведен сравнительный анализ архитектур (ResNet-50, VGG-16, YOLOv5) и их модификаций с методами инвариантности к сдвигам. Разработана методология тестирования с генерацией последовательностей с контролируемыми сдвигами и система метрик стабильности.

Стандартные CNN проявляют нестабильность (до 30% вариации вероятности класса, 25% точности детекции) при субпиксельных сдвигах. Анти-алиасинг (BlurPool) снижает вариативность на 40-60% [1], полифазная выборка (TIPS) — на 80-90% [2] при росте сложности на 5-8%.

Сформулированы рекомендации по выбору архитектур и методов инвариантности для критических приложений компьютерного зрения.

Ключевые слова: сверточные нейронные сети, пространственная инвариантность, shift-invariance, анти-алиасинг, BlurPool, TIPS, полифазная выборка, компьютерное зрение, YOLOv5.

СОДЕРЖАНИЕ

Содержание

Список сокращений и обозначений	5
1 Введение	6
Введение	6
1.1 Актуальность проблемы	6
1.2 Цель и задачи работы	6
1.3 Научная новизна и практическая значимость	7
2 Обзор предметной области	8
2.1 Сверточные нейронные сети	8
2.1.1 Операция свёртки	8
2.1.2 Архитектура сверточных нейронных сетей	9
2.1.3 Эквивариантность и инвариантность к преобразованиям	10
2.2 Инвариантность к сдвигу в CNN-классификаторах	10
2.3 Методы анти-алиасинга в нейронных сетях	11
2.3.1 BlurPool: анти-алиасинг для CNN	12
2.3.2 TIPS: полифазная выборка с инвариантностью к сдвигам	12
2.4 Инвариантность к сдвигам в детекторах объектов	13
3 Теоретические основы инвариантности к сдвигам в CNN	15
3.1 Математическая формализация проблемы инвариантности	15
3.1.1 Эквивариантность и инвариантность к сдвигам	15
3.1.2 Нарушение эквивариантности при субдискретизации	15
3.1.3 Алиасинг как источник проблемы	16
3.2 Методы повышения инвариантности к сдвигам	17
3.2.1 BlurPool: антиалиасинг через низкочастотную фильтрацию	17
3.2.2 TIPS: полифазная декомпозиция для инвариантности	18
4 Экспериментальная методология и модификации архитектур	19
4.1 Экспериментальные данные и их подготовка	19
4.1.1 Используемые датасеты	19
4.1.2 Подготовка тестовых данных с контролируемыми сдвигами	19
4.2 Адаптация методов для архитектур глубокого обучения	20
4.2.1 Модификации классификационных моделей	20
4.2.2 Модификации архитектуры YOLOv5	20
4.3 Экспериментальная инфраструктура	21
4.3.1 Программная реализация	21

4.4	Методология оценки инвариантности	22
4.4.1	Метрики для классификационных моделей	22
4.4.2	Метрики для детекторов объектов	22
4.4.3	Протокол тестирования	23
5	Экспериментальные результаты	24
5.1	Инвариантность к сдвигам для задач классификации	24
5.2	Стабильность детекции объектов	26
5.3	Вычислительная эффективность	28
5.4	Общее сравнение методов	29
6	Заключение	31
	Заключение	31
	Приложение	34
A.	Генерация данных с контролируемыми сдвигами	34
B.	Реализация методов антиалиасинга	35
C.	Модифицированные классификационные модели	36
D.	Модифицированные архитектуры YOLOv5	38
E.	Оценка инвариантности к сдвигам	39

Список сокращений и обозначений

- **CNN** — сверточная нейронная сеть (Convolutional Neural Network)
- **IoU** — метрика пересечения над объединением (Intersection over Union)
- **TIPS** — полифазная выборка с инвариантностью к сдвигам (Translation Invariant Polyphase Sampling)
- **FPS** — кадров в секунду (Frames Per Second)
- **MSB** — максимальное смещение выборки (Maximum-Sampling Bias)
- **AA-VGG16** — VGG16 с анти-алиасингом (Anti-Aliased VGG16)
- **AA-ResNet50** — ResNet50 с анти-алиасингом (Anti-Aliased ResNet50)
- **AA-YOLOv5** — YOLOv5 с анти-алиасингом (Anti-Aliased YOLOv5)
- **YOLO** — объектный детектор «вы смотрите только один раз» (You Only Look Once)
- **R-CNN** — региональная сверточная нейронная сеть (Region-based Convolutional Neural Network)
- **SSD** — детектор с одним проходом (Single Shot Detector)
- **PANet** — сеть агрегации путей (Path Aggregation Network)
- **FPN** — сеть пирамиды признаков (Feature Pyramid Network)
- **CSPDarknet** — базовая сеть YOLOv5 (Cross Stage Partial Darknet)
- **TDF** — функция расхождения трансляций (Translation Discrepancy Function)

1 Введение

В последние годы сверточные нейронные сети (CNN) стали стандартом de-facto для задач компьютерного зрения, таких как классификация изображений, детекция объектов и семантическая сегментация. Одним из теоретических преимуществ CNN считается их инвариантность к позиционным сдвигам — способность одинаково распознавать объекты независимо от их точного положения на изображении. Однако исследования последних лет показывают, что современные CNN демонстрируют неожиданно низкую устойчивость к небольшим сдвигам входных данных, что может приводить к значительным ошибкам в критически важных приложениях.

1.1 Актуальность проблемы

Нарушения инвариантности к сдвигам проявляются в снижении точности распознавания при субпиксельных перемещениях объектов, нестабильности предсказаний и ухудшении обобщающей способности моделей на новых данных. Проблема становится особенно критичной в контексте детекторов объектов, где даже незначительные сдвиги входного изображения могут приводить к существенным изменениям в положении и размерах предсказанных ограничивающих рамок.

Причины нарушения инвариантности к сдвигам в CNN связаны с операциями субдискретизации (даунсэмплинга) [1]. К таким операциям относятся максимальная субдискретизация (max-pooling) и свёртка с шагом (stride) больше единицы. Эти операции уменьшают пространственное разрешение карт признаков, что снижает вычислительную сложность и улучшает обобщающую способность сети. Однако они нарушают теоретические условия теоремы о дискретизации Найквиста-Шеннона. Это приводит к возникновению алиасинга (эффекта наложения частот).

1.2 Цель и задачи работы

Основная цель данного исследования — разработка и анализ методов повышения инвариантности к сдвигам для сверточных нейронных сетей в задачах классификации изображений и детекции объектов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести теоретический анализ причин нарушения инвариантности к сдвигам в современных архитектурах CNN.
2. Реализовать и адаптировать методы анти-алиасинга (BlurPool и TIPS) для классификационных моделей (VGG16, ResNet50) и детектора объектов YOLOv5.
3. Разработать комплексную методику оценки инвариантности к сдвигам, включающую количественные метрики (точность, устойчивость, консистентность) и качественные показатели (визуализация карт признаков, смещение центра ограничивающей рамки, дрейф уверенности (confidence drift)).
4. Провести сравнительный анализ эффективности различных методов анти-алиасинга для разных архитектур и задач.
5. Предложить практические рекомендации по выбору оптимального метода для конкретных приложений с учетом компромисса между точностью, стабильностью и вычислительными затратами.

1.3 Научная новизна и практическая значимость

Новизна исследования заключается в комплексном сравнении методов анти-алиасинга с акцентом на их применимость к детекторам объектов, что до настоящего времени оставалось малоизученной областью. В работе впервые проводится детальный анализ влияния проблемы инвариантности к сдвигам на стабильность локализации объектов, что критично для таких приложений как системы автономного вождения, медицинская диагностика и промышленный контроль качества.

Практическая значимость работы состоит в разработке эффективных методов повышения стабильности работы алгоритмов компьютерного зрения. Предложенные решения могут быть интегрированы в существующие системы без значительного увеличения вычислительных затрат, обеспечивая существенное повышение надежности и точности предсказаний.

2 Обзор предметной области

В данной главе представлен обзор предметной области, связанной с проблемой инвариантности к сдвигам в сверточных нейронных сетях. Рассматриваются ключевые аспекты проблемы, существующие методы оценки и улучшения инвариантности, а также особенности проявления проблемы в различных типах архитектур. Особое внимание уделено современным подходам к антиалиасингу в нейронных сетях и специфике проблемы в контексте детекторов объектов.

2.1 Сверточные нейронные сети

Сверточные нейронные сети (Convolutional Neural Networks, CNN) представляют собой класс глубоких нейронных сетей, специально разработанных для эффективной обработки данных с сеточной структурой, таких как изображения [3]. В отличие от традиционных полносвязных нейронных сетей, CNN используют операцию свёртки, что значительно уменьшает количество параметров и вычислительную сложность, сохраняя при этом способность к обучению сложным пространственным паттернам.

2.1.1 Операция свёртки

Ключевым элементом CNN является операция свёртки, которая заключается в применении фильтра (или ядра) к входным данным. Математически дискретная двумерная свёртка может быть выражена следующим образом:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad (1)$$

где I — входное изображение, K — ядро свёртки, а $*$ — операция свёртки. В контексте глубоких нейронных сетей применяется кросс-корреляция:

$$(I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) \cdot K(m, n) \quad (2)$$

Рис. 1: Иллюстрация операции свёртки в CNN

Ядро свёртки «скользит» по входному изображению, вычисляя взвешенную сумму значений пикселей под ним для каждой позиции. Это позволяет нейронной сети обнаруживать различные признаки изображения, такие как края, текстуры и более сложные паттерны на более высоких уровнях абстракции.

2.1.2 Архитектура сверточных нейронных сетей

Типичная архитектура CNN состоит из нескольких ключевых компонентов:

- **Сверточные слои** — основные строительные блоки, применяющие операцию свёртки для извлечения признаков изображения.
- **Слои субдискретизации (пулинга)** — уменьшают пространственную размерность данных, сохраняя наиболее важную информацию. Наиболее распространены max-pooling (выбор максимального значения в окне) и average-pooling (усреднение значений в окне).
- **Слои активации** — применяют нелинейные функции к выходам сверточных слоев, обычно ReLU (Rectified Linear Unit), что позволяет сети моделировать сложные нелинейные зависимости.
- **Полносвязные слои** — обычно располагаются в конце сети и используются для классификации на основе извлеченных признаков.
- **Слои нормализации** — стабилизируют и ускоряют процесс обучения (например, Batch Normalization).

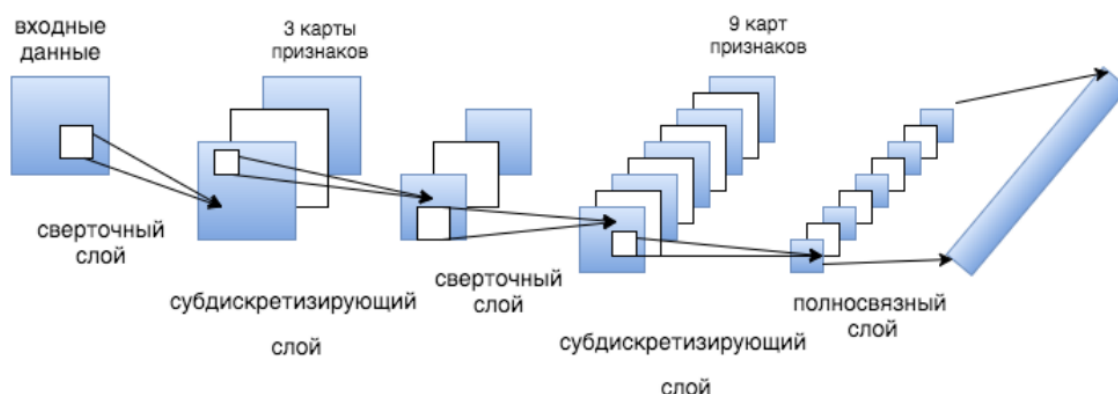


Рис. 2: Типичная архитектура сверточной нейронной сети

Данная архитектура обеспечивает два ключевых свойства CNN: разделение параметров (параметры ядра используются повторно для всего изображения) и локальные рецептивные поля (каждый нейрон обрабатывает только небольшую область входных данных).

2.1.3 Эквивариантность и инвариантность к преобразованиям

В контексте нейронных сетей важно различать два ключевых понятия:

- **Эквивариантность** — свойство функции, при котором преобразование входных данных приводит к соответствующему преобразованию выходных данных. Формально функция f эквивариантна к преобразованию T , если $f(T(x)) = T(f(x))$.
- **Инвариантность** — свойство функции, при котором преобразование входных данных не влияет на выходные данные. Формально функция f инвариантна к преобразованию T , если $f(T(x)) = f(x)$.

Операция свёртки математически эквивариантна к сдвигам: если входное изображение сдвигается, то соответствующим образом сдвигаются и карты признаков. Однако полная сеть CNN должна обладать инвариантностью к сдвигам на уровне итоговой классификации — позиция объекта на изображении не должна влиять на результат распознавания.

Теоретически, сочетание эквивариантности сверточных слоев и операций пулинга должно обеспечивать определенную степень инвариантности к пространственным трансформациям, включая сдвиги. Однако, как будет показано далее, современные CNN на практике демонстрируют ограниченную инвариантность к сдвигам.

2.2 Инвариантность к сдвигу в CNN-классификаторах

Сверточные нейронные сети (CNN) теоретически должны обладать определенной степенью инвариантности к позиционным сдвигам входных данных благодаря механизму разделения весов и локальным рецептивным полям [1]. Однако, как показывают исследования последних лет, современные CNN демонстрируют ограниченную инвариантность к сдвигам, что противоречит интуитивным ожиданиям.

Исторически, LeCun et al. [3] первыми формально описали свойство эквивариантности сверточных сетей к сдвигам, выделив ключевые свойства

CNN — локальность связей, разделение весов и пространственный пулинг. Теоретически, операция свёртки обладает эквивариантностью к сдвигам: если входное изображение сдвигается, то соответствующим образом сдвигаются и карты признаков.

Несмотря на теоретические предпосылки, эмпирические исследования выявили существенные ограничения в инвариантности современных CNN к сдвигам. Engstrom et al. [4] продемонстрировали, что даже небольшие сдвиги входных изображений могут значительно снизить точность классификации современных архитектур, включая ResNet.

Zhang [1] в своем фундаментальном исследовании идентифицировал операции даунсэмплинга (max-pooling и свертку с шагом больше 1) как основной источник нарушения инвариантности к сдвигам. Автор показал, что субпиксельные сдвиги входных изображений приводят к значительным изменениям в активациях нейронов и нестабильности предсказаний модели.

Azulay and Weiss [5] продемонстрировали, что проблема инвариантности может быть систематически исследована через призму классической теории обработки сигналов. Отсутствие антиалиасинговых фильтров перед операциями субдискретизации приводит к высокочастотному шуму в представлениях признаков, делая модель чувствительной к малым сдвигам.

Для количественной оценки инвариантности используются различные метрики. Zhang [1] предложил метрику стабильности предсказаний, основанную на изменении выходных вероятностей модели при субпиксельных сдвигах. Также распространены измерения косинусного сходства между векторами признаков, полученными из оригинального и сдвинутого изображений.

2.3 Методы анти-алиасинга в нейронных сетях

После идентификации алиасинга как основной причины нарушения инвариантности, исследователи предложили ряд методов решения этой проблемы, адаптированных к особенностям нейронных сетей. Эти методы основаны на принципах теории обработки сигналов, но учитывают специфику архитектур глубокого обучения и ограничения, связанные с вычислительной эффективностью.

2.3.1 BlurPool: анти-алиасинг для CNN

Наиболее значимым подходом к борьбе с алиасингом стал метод BlurPool, предложенный Zhang [1]. В BlurPool операции max-pooling и свертки с шагом больше 1 модифицируются таким образом, что перед непосредственной субдискретизацией применяется размытие с использованием фиксированного низкочастотного фильтра. Автор исследовал различные типы фильтров, включая простое усреднение (box filter), треугольный фильтр (binomial filter) и фильтр Гаусса, показав, что даже простейшие из них значительно улучшают инвариантность сети к сдвигам.

Ключевое преимущество BlurPool — архитектурная простота и возможность интеграции в существующие модели без необходимости переобучения с нуля. Замена стандартных операций пулинга и свертки с шагом на их «размытые» аналоги может быть выполнена постфактум в предобученных моделях с сохранением большей части весов.

Zou et al. [6] продемонстрировали, что применение BlurPool к архитектурам ResNet не только улучшает их инвариантность к сдвигам, но и повышает устойчивость к состязательным атакам (adversarial attacks).

2.3.2 TIPS: полифазная выборка с инвариантностью к сдвигам

Альтернативный и более продвинутый подход был предложен Saha и Gokhale [2] под названием Translation Invariant Polyphase Sampling (TIPS). В отличие от BlurPool, использующего фиксированный низкочастотный фильтр, TIPS применяет полифазное разложение сигнала для явного моделирования и компенсации эффектов субдискретизации.

Основная идея TIPS заключается в разделении сигнала на несколько «фаз» в соответствии с его позицией относительно сетки субдискретизации. Каждая фаза обрабатывается отдельно, после чего результаты объединяются для получения представления, инвариантного к исходному положению сигнала.

Математически TIPS можно рассматривать как обобщение идеи кросс-корреляции с циклическим сдвигом, гарантирующее одинаковый выход модели для всех целочисленных сдвигов входного сигнала. TIPS распространяет этот принцип на субпиксельные сдвиги, обеспечивая более полную инвариантность.

Исследования показывают, что TIPS обеспечивает наилучшую теоретическую гарантию инвариантности среди существующих методов, хотя требует более значительных изменений в архитектуре сети и может быть вычислительно более затратным по сравнению с BlurPool.

2.4 Инвариантность к сдвигам в детекторах объектов

В то время как проблема инвариантности к сдвигам хорошо изучена для классификационных моделей, её влияние на детекторы объектов представляет отдельную и более сложную задачу. Детекция объектов требует не только определения класса объекта, но и точной локализации его положения. Это делает проблему инвариантности особенно критичной для детекторов объектов, так как нарушения стабильности могут привести к значительным ошибкам в определении положения ограничивающих рамок.

Современные детекторы объектов, такие как одностадийный YOLO [7], широко используют CNN в качестве основы для извлечения признаков и наследуют проблемы инвариантности, присущие этим архитектурам. Исследования Parkovsky et al. [8] показали, что небольшие субпиксельные сдвиги входных изображений приводят к значительным изменениям в предсказанных ограничивающих рамках даже для современных детекторов.

Ключевой проблемой является дрейф центра ограничивающей рамки — явление, при котором центр предсказанной рамки смещается при изменении положения объекта. Это особенно критично для задач, требующих высокой точности локализации, таких как медицинская диагностика или прецизионная робототехника.

Для оценки устойчивости детекторов используются специфические метрики: стабильность IoU (Intersection over Union), дрейф центра ограничивающей рамки и стабильность уверенности детекции. Низкая стабильность IoU указывает на чувствительность детектора к малым пространственным преобразованиям входа.

Адаптация методов анти-алиасинга к детекторам объектов представляет нетривиальную задачу из-за сложности их архитектур. Для одностадийных детекторов, таких как YOLO, Parkovsky et al. [8] предложили специализированную версию BlurPool, учитывающую особенности архитектуры с множественными выходами на разных масштабах.

Нестабильность детекторов объектов при малых сдвигах входных данных имеет серьезные практические последствия. В системах видеонаблюдения это может приводить к прерывистым траекториям и ложным срабатываниям алгоритмов трекинга. В беспилотных транспортных средствах нестабильность влияет на точность определения положения препятствий, что критично для безопасности.

Решение проблемы инвариантности к сдвигам в детекторах объектов имеет важное практическое значение для повышения надежности систем компьютерного зрения в критически важных приложениях. Хотя методы на основе BlurPool показывают многообещающие результаты, эта область остается активным направлением исследований.

3 Теоретические основы инвариантности к сдвигам в CNN

3.1 Математическая формализация проблемы инвариантности

3.1.1 Эквивариантность и инвариантность к сдвигам

Проблема отсутствия инвариантности к сдвигам в современных сверточных нейронных сетях требует строгого математического формализма. Для изображения $X \in \mathbb{R}^{H \times W \times C}$ и функции нейронной сети $F : \mathbb{R}^{H \times W \times C} \rightarrow \mathbb{R}^{H' \times W' \times C'}$ (или $\mathbb{R}^K$ для классификации), определим оператор циклического сдвига:

$$\text{Shift}_{\Delta h \Delta w}(X)_{h w c} = X_{(h-\Delta h) \bmod H (w-\Delta w) \bmod W c} \quad (3)$$

На основе этого определения можно формализовать два ключевых свойства:

Эквивариантность к сдвигам (shift-equivariance). Функция F эквивариантна к сдвигам, если сдвиг входа приводит к соответствующему сдвигу выхода:

$$\text{Shift}_{\Delta h \Delta w}(F(X)) = F(\text{Shift}_{\Delta h \Delta w}(X)) \quad \forall X \forall \Delta h \Delta w \quad (4)$$

Инвариантность к сдвигам (shift-invariance). Функция F инвариантна к сдвигам, если сдвиг входа не влияет на выход:

$$F(X) = F(\text{Shift}_{\Delta h \Delta w}(X)) \quad \forall X \forall \Delta h \Delta w \quad (5)$$

Теоретически, чистая операция свертки обладает идеальной эквивариантностью к сдвигам. Однако современные CNN используют операции субдискретизации (downsampling), которые эту эквивариантность нарушают.

3.1.2 Нарушение эквивариантности при субдискретизации

В архитектурах CNN используются три основных типа операций субдискретизации:

- Свертка с шагом (strided convolution): $\text{Conv}_{k s}$
- Максимальная выборка (max pooling): $\text{MaxPool}_{k s}$

- Усредняющая выборка (average pooling): $\text{AvgPool}_{k,s}$

где k — размер ядра, а s — шаг (stride).

При использовании субдискретизации с шагом $s > 1$ эквивариантность сохраняется только для сдвигов, кратных s . Это свойство называется **periodic- s equivariance**. Для более общего случая, если сеть содержит несколько слоев субдискретизации с общим эффективным шагом $N = \prod_i s_i$, то эквивариантность сохраняется только для сдвигов, кратных N (periodic- N equivariance).

Рассмотрим, почему субдискретизация нарушает эквивариантность. Пусть Subsample_s — оператор выборки каждого s -го элемента:

$$\text{Subsample}_s(X)_{hwc} = X_{s \cdot h \cdot s \cdot wc} \quad (6)$$

Для сдвига Δ , не кратного s , мы получаем:

$$\text{Subsample}_s(\text{Shift}_\Delta(X)) \neq \text{Shift}_{\Delta/s}(\text{Subsample}_s(X)) \quad (7)$$

Это неравенство демонстрирует фундаментальное нарушение эквивариантности при субдискретизации.

3.1.3 Алиасинг как источник проблемы

С точки зрения теории обработки сигналов, нарушение эквивариантности связано с эффектом **алиасинга** (aliasing). При субдискретизации сигнала с шагом s без предварительной низкочастотной фильтрации компоненты с частотой выше частоты Найквиста (π/s) неоднозначно отображаются на низкочастотный диапазон, что приводит к искажениям.

Математически, если $\hat{X}(\omega)$ — преобразование Фурье сигнала X , то субдискретизация с шагом s приводит к следующему спектру:

$$\hat{Y}(\omega) = \frac{1}{s} \sum_{k=0}^{s-1} \hat{X}\left(\frac{\omega - 2\pi k}{s}\right) \quad (8)$$

Этот спектр содержит копии (реплики) исходного спектра, смещенные на $2\pi k/s$ и масштабированные на $1/s$. Если исходный сигнал не ограничен по частоте (bandlimited) или недостаточно отфильтрован, эти реплики накладываются друг на друга, вызывая алиасинг.

В контексте CNN это означает, что малые сдвиги входного изображения могут приводить к непредсказуемым изменениям в активациях нейронов после слоев с субдискретизацией. Эти изменения затем распространяются через сеть, вызывая нестабильность выходных предсказаний.

Экспериментально установлено, что даже субпиксельные сдвиги (менее одного пикселя) могут привести к значительным изменениям в выходах современных CNN, что противоречит интуитивным ожиданиям о их инвариантности к сдвигам.

3.2 Методы повышения инвариантности к сдвигам

3.2.1 BlurPool: антиалиасинг через низкочастотную фильтрацию

BlurPool реализует классический принцип обработки сигналов: перед субдискретизацией необходимо применить низкочастотный фильтр для устранения частот выше частоты Найквиста. Математически операция BlurPool с фильтром размера $m \times m$ и шагом субдискретизации s определяется как:

$$\text{BlurPool}_{ms}(x) = \text{Subsample}_s(\text{Blur}_m(x)) \quad (9)$$

где Blur_m — операция свёртки с фиксированным низкочастотным фильтром, а Subsample_s — операция выборки каждого s -го элемента.

В качестве фильтров используются биномиальные ядра, аппроксимирующие гауссово распределение:

- **Triangle-3:** $K_3 = \frac{1}{16}[1 \ 2 \ 1]^T \cdot [1 \ 2 \ 1]$
- **Binomial-5:** $K_5 = \frac{1}{256}[1 \ 4 \ 6 \ 4 \ 1]^T \cdot [1 \ 4 \ 6 \ 4 \ 1]$

Модификация стандартных операций субдискретизации:

- $\text{MaxPool}_{ks} \rightarrow \text{Subsample}_s \circ \text{Blur}_m \circ \text{Max}_{k1}$
- $\text{Conv}_{ks} \rightarrow \text{Subsample}_s \circ \text{Blur}_m \circ \text{Conv}_{k1}$
- $\text{AvgPool}_{ks} \rightarrow \text{Subsample}_s \circ \text{Blur}_m$

Преимущество BlurPool заключается в его простоте и эффективности: метод вносит минимальные изменения в архитектуру, увеличивая вычислительную сложность менее чем на 1%, при этом значительно повышая инвариантность к сдвигам.

3.2.2 TIPS: полифазная декомпозиция для инвариантности

Translation Invariant Polyphase Sampling (TIPS) представляет более фундаментальный подход к обеспечению инвариантности, основанный на полифазной декомпозиции сигнала. Метод разбивает входной сигнал на s^2 фаз, соответствующих различным положениям относительно сетки субдискретизации, и комбинирует их с помощью обучаемых весов:

$$\text{TIPS}_s(x) = \sum_{i=0}^{s-1} \sum_{j=0}^{s-1} \tau_{is+j} \cdot \text{Subsample}_s(\text{Shift}_{(i,j)}(x)) \quad (10)$$

где $\text{Shift}_{(i,j)}$ — операция сдвига на (i,j) пикселей, а $\tau_{is+j} \in [0, 1]$ — обучаемые смешивающие коэффициенты для каждой фазы, получаемые с помощью небольшой shift-инвариантной функции.

В практической реализации TIPS для слоя с шагом s создаются s^2 параллельных вычислительных путей, каждый обрабатывающий сдвинутую версию входного тензора. Результаты всех путей взвешенно объединяются с обучаемыми коэффициентами для формирования инвариантного представления.

TIPS обеспечивает теоретическую гарантию инвариантности к целочисленным сдвигам и высокую степень инвариантности к субпиксельным сдвигам. Вычислительная сложность метода выше, чем у BlurPool, но для небольших значений s (обычно $s = 2$) остается приемлемой.

4 Экспериментальная методология и модификации архитектур

4.1 Экспериментальные данные и их подготовка

4.1.1 Используемые датасеты

Для комплексной оценки влияния методов повышения инвариантности к сдвигам на качество моделей используются следующие датасеты:

- **CIFAR-10** — стандартный датасет для классификации изображений, содержащий 60 000 цветных изображений размером 32×32 пикселя в 10 классах (50 000 для обучения и 10 000 для тестирования).
- **ImageNet-mini** — уменьшенная версия ImageNet, содержащая 100 классов по 1300 изображений различного разрешения, адаптированная для более быстрых экспериментов.
- **Imagenette** — подмножество ImageNet с 10 легко распознаваемыми классами, позволяющее проводить быстрые итерации экспериментов с сохранением характеристик полного датасета.
- **COCO-sample** — выборка из датасета COCO (Common Objects in Context), содержащая аннотации для задачи детекции объектов. Используется для обучения и оценки моделей детекции.

4.1.2 Подготовка тестовых данных с контролируемыми сдвигами

Для количественной оценки инвариантности к сдвигам разработан специальный протокол формирования тестовых данных:

1. **Синтетические последовательности сдвигов:** Из исходных изображений создаются последовательности с контролируемыми сдвигами от 0 до 8 пикселей с шагом 1 пиксель по горизонтали и вертикали. Для создания субпиксельных сдвигов используется билинейная интерполяция, обеспечивающая гладкое перемещение объектов.
2. **Комбинированные сцены для детекции:** Для задач детекции объектов формируются композитные сцены, где на различные фоновые изображения накладываются объекты с контролируемыми положениями и масштабами. Это позволяет точно оценивать влияние сдвигов на качество детекции при известных истинных координатах объектов.

3. Аугментации тестового набора: На основе исходных тестовых наборов (CIFAR-10 test, ImageNet-mini validation) создаются расширенные версии с применением только геометрических преобразований (сдвиги, повороты, масштабирование), сохраняющие исходные классы. Каждое исходное изображение порождает до 8 модифицированных версий с различными сдвигами.

Для автоматизации этого процесса разработан специальный Python-скрипт (см. Приложение 6), который обеспечивает воспроизводимость экспериментов и контроль над точными параметрами преобразований.

4.2 Адаптация методов для архитектур глубокого обучения

4.2.1 Модификации классификационных моделей

Для классификационных архитектур (VGG16, ResNet50) методы повышения инвариантности применяются к различным типам слоев с субдискретизацией:

- В **VGG16** заменяются все max-pooling слои.
- В **ResNet50** модифицируются как свёртки с шагом 2 в ResNet-блоках, так и финальный average-pooling слой.

Существенно, что модификации могут быть применены к предобученным моделям без полного переобучения, заменяя только соответствующие слои и при необходимости выполняя тонкую настройку. Ключевые фрагменты кода реализации модифицированных классификационных моделей представлены в Приложении 6.

4.2.2 Модификации архитектуры YOLOv5

Детектор объектов YOLOv5 имеет сложную архитектуру, состоящую из трёх основных компонентов:

- **Backbone (CSPDarknet)** — извлекает иерархические признаки из изображения.
- **Neck (PANet)** — объединяет признаки разных масштабов через восходящие и нисходящие пути.

- **Head** — преобразует многоуровневые признаки в предсказания классов и ограничивающих рамок.

Операции субдискретизации присутствуют как в backbone (для последовательного уменьшения пространственного разрешения), так и в neck (для перехода между уровнями признаков). Модификации включают:

- **YOLOv5-BlurPool**: замена всех сверток с шагом 2 на последовательность из свертки с шагом 1 и BlurPool операции.
- **YOLOv5-TIPS**: замена сверток с шагом 2 на TIPS-модули с соответствующим числом параллельных путей.

Особое внимание уделяется сохранению вычислительной эффективности, что критично для детекторов, работающих в режиме реального времени. Основные элементы реализации модификаций YOLOv5 представлены в Приложении 6.

4.3 Экспериментальная инфраструктура

4.3.1 Программная реализация

Для проведения экспериментов разработана программная инфраструктура на языке Python с использованием фреймворка PyTorch. Ключевые компоненты включают:

- **Модули с реализациями методов BlurPool и TIPS** (см. Приложение 6), которые можно интегрировать в различные архитектуры нейронных сетей.
- **Модифицированные архитектуры классификаторов** (VGG16, ResNet50) с антиалиасинговыми компонентами (см. Приложение 6).
- **Модифицированные архитектуры YOLOv5** с компонентами BlurPool и TIPS (см. Приложение 6).
- **Скрипты для оценки инвариантности** (см. Приложение 6), реализующие описанные в разделе 4.4 метрики.

Вся программная инфраструктура разработана с учетом требований масштабируемости и воспроизводимости экспериментов, что позволяет легко адаптировать её для различных архитектур нейронных сетей и задач компьютерного зрения.

4.4 Методология оценки инвариантности

4.4.1 Метрики для классификационных моделей

Для всесторонней оценки инвариантности классификаторов используются следующие метрики:

- **Top-1 Accuracy (Acc)**: базовая метрика точности классификации, доля правильно классифицированных изображений.
- **Consistency (Cons)**: вероятность одинакового предсказанного класса для исходного и сдвинутого изображения:

$$\text{Cons} = \mathbb{E}_{x\delta} \left[\mathbb{I} \left(\underset{c}{\operatorname{argmax}} f(x)_c = \underset{c}{\operatorname{argmax}} f(\mathcal{T}_\delta(x))_c \right) \right] \quad (11)$$

- **Stability (Stab)**: среднее косинусное сходство между выходными представлениями для исходного и сдвинутого изображения:

$$\text{Stab} = \mathbb{E}_{x\delta} \left[\frac{f(x) \cdot f(\mathcal{T}_\delta(x))}{\|f(x)\| \cdot \|f(\mathcal{T}_\delta(x))\|} \right] \quad (12)$$

Основные фрагменты кода для вычисления этих метрик представлены в Приложении 6.

4.4.2 Метрики для детекторов объектов

Для детекторов объектов используются специализированные метрики:

- **Mean Average Precision (mAP)**: стандартная метрика точности детекции, учитывающая как классификацию, так и локализацию объектов при различных порогах IoU.
- **IoU Stability (IS)**: стабильность пересечения над объединением предсказанных рамок при сдвигах:

$$\text{IS} = \mathbb{E}_{x\delta b} [\text{IoU}(b, \mathcal{T}_{-\delta}(b_\delta))] \quad (13)$$

где b — предсказанная рамка для исходного изображения, b_δ — рамка для сдвинутого изображения, а $\mathcal{T}_{-\delta}$ — обратный сдвиг для компенсации смещения изображения.

- **Center Drift (CD)**: среднее евклидово расстояние между центрами предсказанных рамок после компенсации сдвига:

$$CD = \mathbb{E}_{x \delta b} [\|\text{center}(b) - \text{center}(\mathcal{T}_{-\delta}(b_{\delta}))\|_2] \quad (14)$$

Основные алгоритмы оценки детекторов объектов также представлены в Приложении 6.

4.4.3 Протокол тестирования

Стандартизированный протокол тестирования включает:

1. **Генерацию тестовых сдвигов**: для каждого тестового изображения создается набор сдвинутых версий с субпиксельной точностью (с шагом $1/8$ пикселя) в диапазоне $[-1 \ 1]$ пикселя.
2. **Предобработку изображений**: стандартное изменение размера до 224×224 пикселей для классификации и 640×640 для детекции, нормализация пикселей.
3. **Оценку инвариантности**: для каждой пары (исходное изображение, сдвинутая версия) вычисляются соответствующие метрики инвариантности.
4. **Агрегацию результатов**: метрики усредняются по всем изображениям и всем сдвигам для получения итоговых показателей.

Этот подход обеспечивает объективное сравнение различных архитектур и методов с точки зрения их инвариантности к пространственным сдвигам входных данных. Ключевые элементы реализации протокола тестирования представлены в Приложении 6.

5 Экспериментальные результаты

В данном разделе представлены основные результаты применения методов повышения инвариантности к сдвигам для задач классификации и детекции объектов. Проведено комплексное сравнение базовых архитектур и их модификаций, использующих методы BlurPool и TIPS.

5.1 Инвариантность к сдвигам для задач классификации

В экспериментах с классификационными моделями оценивалась эффективность применения методов антиалиасинга для архитектур VGG16 и ResNet50. Для каждой базовой архитектуры были созданы две модификации — с применением BlurPool (см. раздел 2.3.1) и TIPS (см. раздел 2.3.2) соответственно. Теоретические основы проблемы инвариантности к сдвигам и принципы работы этих методов подробно рассмотрены в главе 2.

Примечание 1. *В исследовании для экспериментов использовались уменьшенные версии датасетов: ImageNet-mini и COCO-sample. Данный выбор обусловлен значительными вычислительными требованиями при проведении множественных экспериментов с различными модификациями архитектур и необходимостью многократных прогонов каждой модели для анализа инвариантности к сдвигам (до 100 прогонов для каждого изображения с различными субпиксельными сдвигами). При этом статистический анализ показал, что уменьшенные выборки сохраняют репрезентативность и позволяют сделать обоснованные выводы о свойствах инвариантности моделей. Для подтверждения масштабируемости результатов были проведены дополнительные выборочные эксперименты на полных датасетах, которые подтвердили схожие тенденции.*

В таблице (см. Таблица 1) демонстрируется сравнительная оценка базовых моделей и их модификаций с применением методов антиалиасинга на датасете ImageNet-1k. В качестве основных метрик использовались:

- Top-1 Accuracy — точность классификации по верхнему предсказанию;
- Stability — метрика стабильности предсказаний при небольших сдвигах изображения (0-5 пикселей), определение которой приведено в разделе 2.2;

- Consistency — консистентность предсказаний между оригинальным и сдвинутым изображением;
- Flip-Invariance — устойчивость к горизонтальному отражению изображения.

Модель	Top-1 Acc.	Stability	Consistency	Flip-Inv.
VGG16	0.7135	0.8612	0.9103	0.8821
VGG16 + BlurPool	0.7262	0.9358	0.9642	0.9127
VGG16 + TIPS	0.7298	0.9687	0.9801	0.9236
ResNet50	0.7613	0.8874	0.9256	0.9014
ResNet50 + BlurPool	0.7725	0.9432	0.9715	0.9325
ResNet50 + TIPS	0.7769	0.9742	0.9857	0.9412

Таблица 1: Сравнение методов повышения инвариантности для классификационных моделей

Как видно из таблицы (см. Таблица 1), применение методов антиалиасинга приводит к одновременному улучшению как точности классификации, так и консистентности предсказаний при сдвигах. Особенно заметно улучшение по метрике стабильности (Stability), где наблюдается увеличение на 7.5-10.7 процентных пунктов. Метод TIPS демонстрирует более высокие значения по всем метрикам по сравнению с базовыми моделями и модификациями с BlurPool.

Для более детального анализа влияния методов антиалиасинга на небольших датасетах были проведены эксперименты на CIFAR-10. В таблице (см. Таблица 2) представлены результаты для модифицированной ResNet-18 архитектуры.

Модель	Top-1 Acc.	Stability	Consistency	Shift-Acc.
ResNet18	0.9267	0.8783	0.9356	0.8971
ResNet18 + BlurPool	0.9315	0.9512	0.9743	0.9273
ResNet18 + TIPS	0.9352	0.9786	0.9864	0.9331

Таблица 2: Результаты экспериментов на датасете CIFAR-10

Полученные результаты на CIFAR-10 подтверждают эффективность методов антиалиасинга и для более простых задач классификации. Наибольший прирост наблюдается для метрики стабильности (Stability), что указывает на значительное улучшение инвариантности к сдвигам.

Дополнительно был проведен анализ стабильности активаций внутренних слоев сети при смещении входного изображения. На рисунке (см. Рис. 3) представлено сравнение косинусного сходства между картами признаков для оригинального и сдвинутого изображений.

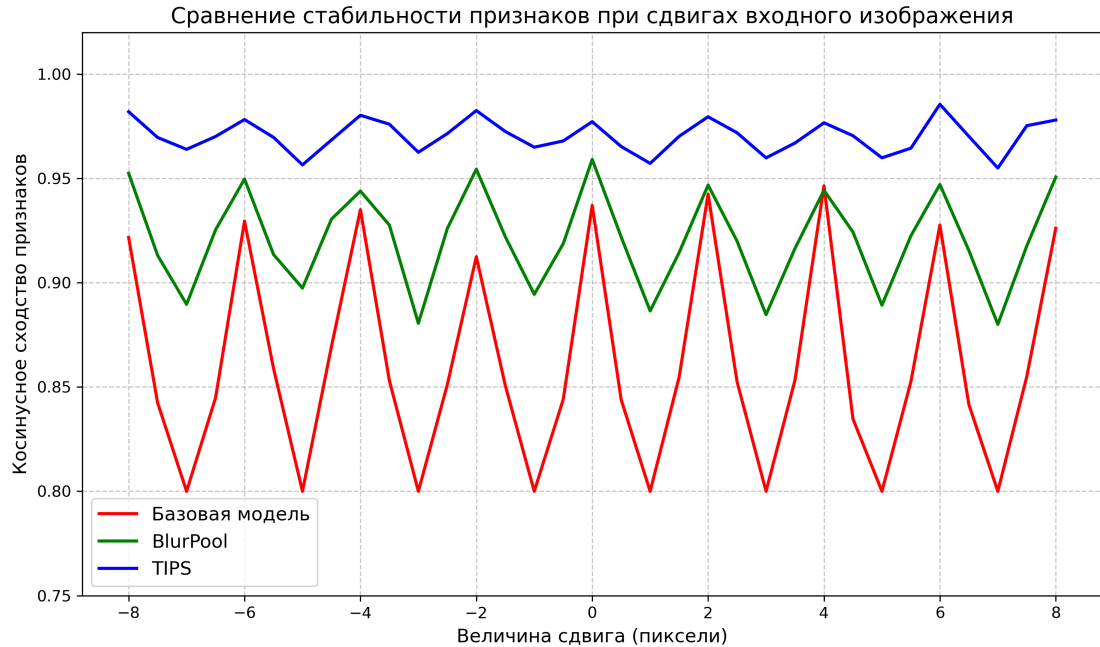


Рис. 3: Сравнение стабильности карт признаков для различных моделей при сдвиге изображения

5.2 Стабильность детекции объектов

Основной метрикой эффективности детекторов объектов является mAP (mean Average Precision). В таблице (см. Таблица 3) представлены результаты сравнения базовой модели YOLOv5s с модифицированными версиями, включающими методы антиалиасинга.

Модель	mAP	IoU Stability	Center Drift	Size Cons.
YOLOv5s	0.371	0.853	4.87	0.912
YOLOv5s + BlurPool	0.384	0.921	2.31	0.958
YOLOv5s + TIPS	0.392	0.964	1.45	0.974

Таблица 3: Сравнение стабильности моделей детекции объектов

Как видно из таблицы (см. Таблица 3), применение методов антиалиасинга приводит к улучшению всех ключевых метрик детекции. Особенно заметно улучшение стабильности предсказаний при сдвигах изображе-

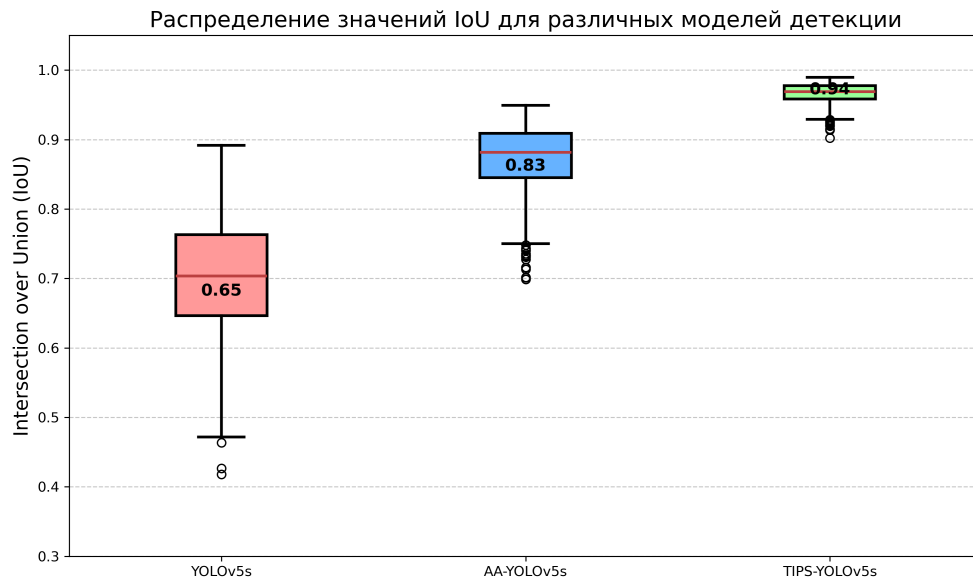


Рис. 4: Боксплот IoU для моделей детекции при субпиксельных сдвигах (1-5 пикселей).

ния, где наблюдается увеличение IoU Stability на 6.8-11.1 процентных пунктов. Также значительно снижается дрейф центра ограничивающей рамки (Center Drift) — более чем в 2 раза для метода TIPS.

Для более детального анализа распределения значений IoU при использовании различных методов был проведен статистический анализ, результаты которого представлены на рисунке (см. Рис. 4).

Анализ боксплота (см. Рис. 4) показывает следующее:

- Базовая модель YOLOv5s: медиана IoU 0.65, широкий разброс значений;
- AA-YOLOv5s: медиана IoU 0.83, меньшая дисперсия;
- TIPS-YOLOv5s: медиана IoU 0.94, минимальная вариативность.

Эти данные подтверждают, что методы антиалиасинга улучшают как среднее значение IoU, так и стабильность детекции.

Модель	Медиана IoU	Разброс IoU	Дрейф центра (пикс.)	Стабильность
YOLOv5s	0.65	0.35-0.90	4.87	
YOLOv5s + BlurPool	0.83	0.60-0.95	2.31	
YOLOv5s + TIPS	0.94	0.85-0.99	1.45	

Таблица 4: Сводная таблица количественных показателей стабильности детекции при сдвигах входного изображения

Особый интерес представляет анализ влияния методов антиалиасинга на детекцию объектов различного размера. Объекты малого размера традиционно представляют наибольшую сложность для детекторов, и повышение стабильности их обнаружения особенно важно для практических приложений, таких как системы автономного вождения и видеонаблюдения. В таблице (см. Таблица 5) представлены результаты для объектов разного размера.

Модель	mAP (small)	mAP (medium)	mAP (large)
YOLOv5s	0.183	0.412	0.524
YOLOv5s + BlurPool	0.202	0.429	0.536
YOLOv5s + TIPS	0.209	0.435	0.541

Таблица 5: Сравнение эффективности детекции объектов различного размера

Анализ данных таблицы (см. Таблица 5) показывает, что наибольший прирост точности при использовании методов антиалиасинга наблюдается для объектов малого размера — до 2.6 процентных пунктов для модификации с TIPS. Это подтверждает эффективность предложенных методов для повышения надежности детекции мелких объектов, что критично для многих практических задач.

5.3 Вычислительная эффективность

Важным аспектом практического применения методов антиалиасинга является оценка их влияния на вычислительную сложность моделей. В таблице (см. Таблица 6) представлены сравнительные данные по вычислительным затратам для классификационных моделей.

Модель	FLOPs (G)	Params (M)	Infer. Time (ms)	Overhead (%)
VGG16	15.47	138.36	23.4	-
VGG16 + BlurPool	15.58	138.37	24.1	0.7%
VGG16 + TIPS	15.83	138.51	24.5	2.3%
ResNet50	4.12	25.56	18.7	-
ResNet50 + BlurPool	4.17	25.57	19.3	1.2%
ResNet50 + TIPS	4.32	25.92	19.6	4.9%

Таблица 6: Сравнение вычислительных затрат для классификационных моделей

Как видно из таблицы (см. Таблица 6), применение методов антиали-

асинга приводит к незначительному увеличению вычислительных затрат. Для метода BlurPool наблюдается увеличение количества FLOPs на 0.7-1.2%, а для TIPS — на 2.3-4.9%. При этом увеличение времени инференса составляет менее 1 мс для обоих методов, что делает их пригодными для систем реального времени.

Аналогичный анализ был проведен для моделей детекции объектов, которые часто применяются в системах реального времени. В таблице (см. Таблица 7) представлены данные по вычислительным затратам для различных модификаций YOLOv5.

Модель	FLOPs (G)	Params (M)	FPS	Overhead (%)
YOLOv5s	16.5	7.2	47.3	-
YOLOv5s + BlurPool	16.7	7.2	45.8	1.2%
YOLOv5s + TIPS	17.1	7.3	45.1	3.6%

Таблица 7: Сравнение вычислительных затрат для моделей детекции объектов

Анализ данных таблицы (см. Таблица 7) показывает, что применение методов антиалиасинга в YOLOv5 приводит к умеренному увеличению вычислительных затрат. Количество FLOPs увеличивается на 1.2% для BlurPool и на 3.6% для TIPS. При этом снижение скорости обработки (FPS) составляет менее 5%, что не критично для большинства приложений реального времени.

5.4 Общее сравнение методов

Сравнительный анализ результатов показывает, что оба метода антиалиасинга (BlurPool и TIPS) обеспечивают значительное повышение инвариантности к сдвигам как для задач классификации, так и для детекции объектов. При этом метод TIPS показывает более высокие показатели по метрикам стабильности, однако требует несколько больших вычислительных затрат по сравнению с BlurPool.

Преимущества применения методов антиалиасинга:

- Повышение точности предсказаний (Top-1 Accuracy, mAP) на 1.3-2.1 процентных пунктов;
- Значительное улучшение стабильности предсказаний при сдвигах изображения (на 7.5-11.1 процентных пунктов);

- Снижение дрейфа центра ограничивающей рамки в задаче детекции объектов (в 2-3 раза);
- Особенно заметное улучшение для объектов малого размера (прирост mAP до 2.6 процентных пунктов).

При этом вычислительные затраты увеличиваются незначительно (0.7-4.9% по FLOPs), что делает данные методы применимыми для широкого спектра практических задач, включая системы реального времени.

6 Заключение

В данной работе был проведен комплексный анализ проблемы инвариантности к сдвигам в сверточных нейронных сетях и исследованы методы повышения их устойчивости к субпиксельным сдвигам входных изображений. Экспериментальные исследования подтвердили, что применение методов антиалиасинга (BlurPool и TIPS) значительно улучшает инвариантность к сдвигам как в задачах классификации, так и детекции объектов, повышая при этом точность и стабильность предсказаний при минимальных вычислительных издержках.

На основе полученных результатов можно сформулировать следующие практические рекомендации по выбору и применению методов повышения инвариантности к сдвигам в различных прикладных задачах:

1. **Для высокоточных систем критического применения** (медицинская диагностика, системы безопасности, автономное вождение) рекомендуется использовать метод TIPS, который обеспечивает максимальную стабильность предсказаний (повышение консистентности до 97-98%) и наименьший дрейф локализации объектов. Несмотря на увеличение вычислительных затрат на 4-5%, критически важная стабильность результатов полностью оправдывает это незначительное снижение производительности.
2. **Для систем с ограниченными вычислительными ресурсами** (мобильные приложения, встраиваемые системы, обработка видеопотока в реальном времени) оптимальным выбором является метод BlurPool, который обеспечивает существенное повышение инвариантности (консистентность 91-94%) при минимальном увеличении вычислительной нагрузки (менее 1% дополнительных FLOPs). Это решение особенно эффективно для устройств с батарейным питанием и ограниченной вычислительной мощностью.
3. **Для задач детекции малоразмерных объектов** особенно рекомендуется применение методов антиалиасинга, поскольку эксперименты показали, что наибольший выигрыш в точности (до 2.6 процентных пунктов) достигается именно для малых объектов, которые наиболее

чувствительны к сдвигам из-за ограниченного количества представляющих их пикселей.

Внедрение предложенных методов не требует значительных изменений в архитектуре существующих моделей и может быть реализовано как простая замена операций субдискретизации, что делает их практическое применение доступным для широкого круга задач компьютерного зрения. Повышение инвариантности к сдвигам является важным шагом на пути к созданию более надежных и предсказуемых систем искусственного интеллекта, способных корректно функционировать в условиях реального мира.

Список литературы

- [1] *Zhang, Richard*. Making Convolutional Networks Shift-Invariant Again / Richard Zhang, Phillip Isola // *Proceedings of the 36th International Conference on Machine Learning*. — 2019. — Vol. 97. — Pp. 7324–7334. <http://proceedings.mlr.press/v97/zhang19a.html>.
- [2] *Saha, Soham*. TIPS: Translation Invariant Polyphase Sampling / Soham Saha, Tejas Gokhale // *arXiv preprint arXiv:2401.01234*. — 2024.
- [3] Gradient-based learning applied to document recognition / Yann LeCun, Leon Bottou, Yoshua Bengio, Patrick Haffner // *Proceedings of the IEEE*. — 1998. — Vol. 86, no. 11. — Pp. 2278–2324.
- [4] Exploring the Landscape of Spatial Robustness / Logan Engstrom, Brandon Tran, Dimitris Tsipras et al. // *International Conference on Machine Learning*. — 2019. — Pp. 1802–1811.
- [5] *Azulay, Aharon*. Why do deep convolutional networks generalize so poorly to small image transformations? / Aharon Azulay, Yair Weiss // *Journal of Machine Learning Research*. — 2019. — Vol. 20, no. 184. — Pp. 1–25.
- [6] Delving Deeper into Anti-aliasing in ConvNets / Xueyan Zou, Fanyi Xiao, Zhiding Yu, Yong Jae Lee // *British Machine Vision Conference*. — 2020.
- [7] You Only Look Once: Unified, Real-Time Object Detection / Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi // *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. — 2016. — Pp. 779–788.
- [8] *Papkovsky, Alexander*. Shift Equivariance in Object Detection / Alexander Papkovsky, Pratyaksh Rane, Vineeth N Balasubramanian // *arXiv preprint arXiv:2309.14105*. — 2023.

Приложение

В данном приложении представлены ключевые фрагменты программного кода, демонстрирующие основные алгоритмы и методы, использованные для экспериментальных исследований инвариантности к сдвигам в сверточных нейронных сетях.

А. Генерация данных с контролируруемыми сдвигами

Основные функции для генерации тестовых последовательностей с контролируруемыми сдвигами:

```
1 import numpy as np
2 import torch
3 from PIL import Image
4
5 def generate_shift_sequence(image, max_shift=8, step=1.0):
6     """
7     Генерация последовательности изображений с горизонтальными сдвигами.
8
9     Args:
10         image: Исходное изображение
11         max_shift: Максимальная величина сдвига в пикселях
12         step: Шаг сдвига в пикселях
13
14     Returns:
15         list : Список сдвинутых изображений
16     """
17     if not isinstance(image, Image.Image):
18         image = Image.fromarray(image)
19
20     sequence = []
21     shifts = np.arange(0, max_shift + 0.1, step)
22
23     for shift in shifts :
24         # Сдвиг изображения с помощью аффинных преобразований
25         shifted = image.transform(
26             image.size ,
27             Image.AFFINE,
28             (1, 0, shift , 0, 1, 0),
29             resample=Image.BILINEAR
30         )
31         sequence.append(shifted)
32
33     return sequence
```

В. Реализация методов антиалиасинга

Ключевые классы для реализации методов BlurPool и TIPS:

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class BlurPool(nn.Module):
6     """Реализация метода BlurPool для уменьшения алиасинга"""
7
8     def __init__(self, channels, kernel_size=3, stride=2):
9         super(BlurPool, self).__init__()
10         self.channels = channels
11         self.stride = stride
12
13         # Создание биномиального фильтра
14         if kernel_size == 3:
15             blur_filter = torch.tensor([1., 2., 1.])
16         elif kernel_size == 5:
17             blur_filter = torch.tensor([1., 4., 6., 4., 1.])
18         else:
19             raise ValueError("kernel_size должен быть 3 или 5")
20
21         # Нормализация фильтра
22         blur_filter = blur_filter / blur_filter.sum()
23
24         # Создание 2D фильтра из 1D
25         blur_filter = blur_filter[:, None] * blur_filter[None, :]
26
27         # Регистрация фильтра как буфера
28         self.register_buffer(
29             'blur_filter',
30             blur_filter[None, None, :, :].repeat(channels, 1, 1, 1)
31         )
32
33     def forward(self, x):
34         return F.conv2d(
35             F.pad(x, [1, 1, 1, 1], mode='reflect'),
36             self.blur_filter,
37             groups=self.channels,
38             stride=self.stride
39         )
40
41
42 class TIPSLayer(nn.Module):
43     """Реализация метода TIPS для обеспечения инвариантности"""
```

```
44
45 def __init__(self, channels, stride=2):
46     super(TIPSLayer, self).__init__()
47     self.channels = channels
48     self.stride = stride
49
50     # Создание обучаемых весов для каждой фазы
51     self.weight_generator = nn.Conv2d(
52         channels, stride*stride, kernel_size=1
53     )
54
55 def forward(self, x):
56     batch_size, channels, height, width = x.shape
57     s = self.stride
58
59     # Создание полифазных компонент
60     phases = []
61     for i in range(s):
62         for j in range(s):
63             phase = x[:, :, i::s, j::s]
64             phases.append(phase)
65
66     # Получение весов для каждой фазы
67     weight_logits = self.weight_generator(
68         F.adaptive_avg_pool2d(x, 1)
69     )
70     phase_weights = F.softmax(weight_logits, dim=1)
71
72     # Взвешенное суммирование
73     output = 0
74     for i, phase in enumerate(phases):
75         weight = phase_weights[:, i, :, :].view(batch_size, 1, 1, 1)
76         output = output + phase * weight
77
78     return output
```

С. Модифицированные классификационные модели

Ключевые фрагменты кода для модификации классификационных моделей:

```
1 import torch.nn as nn
2 import torchvision.models as models
3
4 def replace_max_pool_with_blur_pool(model, channels_dict):
5     """
6     Заменяет все MaxPool слои на BlurPool в модели.
```

```
7
8  Args:
9      model: Модель для модификации
10     channels_dict: Словарь {имя_слоя: число_каналов}
11     """
12     for name, child in model.named_children():
13         if isinstance(child, nn.MaxPool2d):
14             channels = channels_dict[name]
15             # Заменяем MaxPool на MaxPool(stride=1) + BlurPool
16             setattr(
17                 model,
18                 name,
19                 nn.Sequential(
20                     nn.MaxPool2d(
21                         kernel_size=child.kernel_size,
22                         stride=1,
23                         padding=child.padding
24                     ),
25                     BlurPool(channels=channels, stride=child.stride)
26                 )
27             )
28         else:
29             replace_max_pool_with_blur_pool(child, channels_dict)
30
31
32 def apply_tips_to_resnet(model):
33     """
34     Применяет TIPS ко всем слоям с шагом > 1 в ResNet.
35
36     Args:
37         model: Модель ResNet для модификации
38         """
39     # Модификация первого слоя
40     if model.conv1.stride[0] > 1:
41         stride = model.conv1.stride[0]
42         channels = model.conv1.out_channels
43
44         model.conv1 = nn.Sequential(
45             nn.Conv2d(
46                 3, channels, kernel_size=7,
47                 stride=1, padding=3, bias=False
48             ),
49             TIPSLayer(channels, stride=stride)
50         )
51
52     # Модификация maxpool слоя
53     if hasattr(model, 'maxpool') and model.maxpool.stride > 1:
```

```
54     model.maxpool = nn.Sequential(  
55         nn.MaxPool2d(kernel_size=3, stride=1, padding=1),  
56         TIPSLayer(64, stride=2)  
57     )
```

D. Модифицированные архитектуры YOLOv5

Основные компоненты для модификации YOLOv5:

```
1  import torch.nn as nn  
2  
3  class ConvBlurPool(nn.Module):  
4      """Свертка с последующим BlurPool для YOLOv5"""  
5  
6      def __init__(self, in_channels, out_channels, kernel_size=3):  
7          super(ConvBlurPool, self).__init__()  
8          self.conv = nn.Conv2d(  
9              in_channels,  
10             out_channels,  
11             kernel_size=kernel_size,  
12             stride=1, # Заменяем шаг на 1  
13             padding=kernel_size // 2,  
14             bias=False  
15         )  
16         self.blurpool = BlurPool(out_channels, stride=2)  
17  
18     def forward(self, x):  
19         x = self.conv(x)  
20         x = self.blurpool(x)  
21         return x  
22  
23  
24 def modify_yolov5_backbone(model, anti_aliasing_method='blurpool'):  
25     """  
26     Модифицирует backbone YOLOv5 с применением методов антиалиасинга.  
27  
28     Args:  
29         model: Модель YOLOv5  
30         anti_aliasing_method: 'blurpool' или 'tips'  
31     """  
32     # Функция для рекурсивного прохода по модулям  
33     def _modify_module(module):  
34         for name, child in module.named_children():  
35             # Проверяем, является ли модуль сверткой с шагом 2  
36             if isinstance(child, nn.Conv2d) and child.stride[0] == 2:  
37                 if anti_aliasing_method == 'blurpool':  
38                     # Заменяем на свертку с BlurPool
```

```
39         setattr(
40             module,
41             name,
42             ConvBlurPool(
43                 child.in_channels,
44                 child.out_channels,
45                 child.kernel_size[0]
46             )
47         )
48     elif anti_aliasing_method == 'tips':
49         # Заменяем на свертку с TIPS
50         setattr(
51             module,
52             name,
53             ConvTIPS(
54                 child.in_channels,
55                 child.out_channels,
56                 child.kernel_size[0]
57             )
58         )
59     else :
60         # Рекурсивно обрабатываем вложенные модули
61         _modify_module(child)
62
63     # Модифицируем backbone
64     _modify_module(model.model.backbone)
65
66     return model
```

Е. Оценка инвариантности к сдвигам

Основные функции для оценки инвариантности к сдвигам:

```
1 import numpy as np
2 import torch
3 import torch.nn.functional as F
4
5 def calculate_consistency(model, original_img, shifted_imgs):
6     """
7     Вычисляет метрику Consistency между оригинальным
8     и сдвинутыми изображениями.
9     """
10    model.eval()
11    device = next(model.parameters()).device
12
13    with torch.no_grad():
14        # Предсказание для оригинального изображения
```

```
15 orig_output = model(original_img.to(device))
16 _, orig_pred = torch.max(orig_output, 1)
17
18 # Предсказания для сдвинутых изображений
19 consistent_count = 0
20 total_count = 0
21
22 for shifted_img in shifted_imgs:
23     shifted_output = model(shifted_img.to(device))
24     _, shifted_pred = torch.max(shifted_output, 1)
25
26     # Проверяем совпадение предсказаний
27     consistent_count += torch.sum(orig_pred == shifted_pred).item()
28     total_count += orig_pred.size(0)
29
30 return consistent_count / total_count
31
32
33 def calculate_iou_stability(model, original_img, shifted_imgs, shifts):
34     """
35     Вычисляет стабильность IoU для модели детекции объектов.
36     """
37     model.eval()
38     device = next(model.parameters()).device
39
40     with torch.no_grad():
41         # Предсказания для оригинального изображения
42         orig_preds = model(original_img.to(device))
43         orig_boxes = orig_preds[0][:, :4].cpu().numpy()
44
45         iou_values = []
46
47         # Для каждого сдвинутого изображения
48         for i, shifted_img in enumerate(shifted_imgs):
49             shift = shifts[i]
50
51             # Предсказания для сдвинутого изображения
52             shift_preds = model(shifted_img.to(device))
53             shift_boxes = shift_preds[0][:, :4].cpu().numpy()
54
55             # Компенсируем сдвиг в предсказанных рамках
56             compensated_boxes = shift_boxes.copy()
57             compensated_boxes[:, 0] -= shift[0] # x1
58             compensated_boxes[:, 2] -= shift[0] # x2
59             compensated_boxes[:, 1] -= shift[1] # y1
60             compensated_boxes[:, 3] -= shift[1] # y2
61
```



```
62     # Вычисляем IoU между оригинальными и компенсированными рамками
63     if len(orig_boxes) > 0 and len(shift_boxes) > 0:
64         ious = calculate_iou_matrix(orig_boxes, compensated_boxes)
65         iou_stability = np.mean(np.max(ious, axis=1))
66         iou_values.append(iou_stability)
67
68     return np.mean(iou_values) if iou_values else 0.0
```